

src\case_studies\F_vtol\altitude_pd_controller.py

Generated: 2026-02-20 21:11:28

altitude_pd_controller.py

src\case_studies\F_vtol\altitude_pd_controller.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from case_studies import common
import case_studies.F_vtol.params as P
from case_studies.control.pd import PD

class AltitudeControllerPD(common.ControllerBase):
    def __init__(self):
        # tuning parameters
        p1 = -0.2
        p2 = -0.3

        # system parameters
        b0 = P.tf_lon_num[-1]
        a1, a0 = P.tf_lon_den[-2:]

        # desired characteristic equation parameters
        # des_CE:  $s^2 + \alpha_1 s + \alpha_0 = s^2 + (a_1 + b_0 k_d)s + (a_0 + b_0 k_p)$ 
        des_CE = np.poly([p1, p2])
        alpha1, alpha0 = des_CE[-2:]

        # find gains
        kp = (alpha0 - a0) / b0
        kd = (alpha1 - a1) / b0
        print(f"{kp = :.2f}, {kd = :.2f}")

        self.h_pd = PD(kp, kd)
        self.F_eq = P.F_eq

    def update_with_state(self, r, x):
        # unpack references and states
        h_ref = r[1]
        h = x[1]
        hdot = x[4]

        # altitude (modified) PD
        F_tilde = self.h_pd.update_modified(h_ref, h, hdot)
        F = F_tilde + self.F_eq
        tau = 0.0 # not used in altitude controller
        u_FT = np.array([F, tau])

        # convert force-torque to motor forces
        u = P.mixer @ u_FT
        return u
```